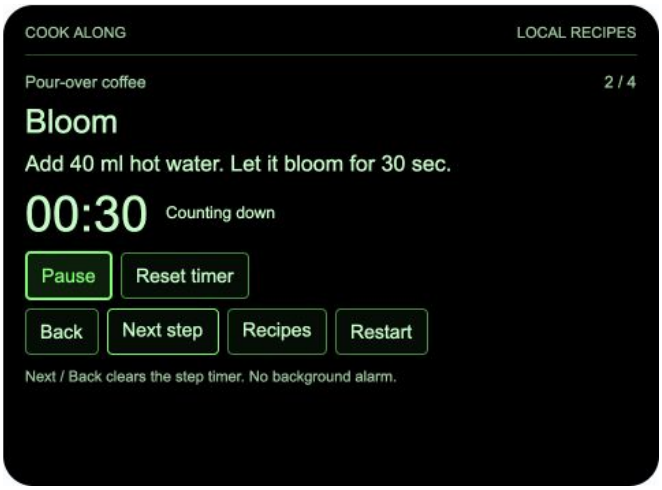
01 / CHOOSE YOUR STARTING POINT

Build your first Rokid AIUI demo

Run a starter. Change one useful thing with the DevKit.
Show your own working flow at the workshop.

Cook Along

A local recipe guide with step navigation and a pause/resume timer. No application API or key.

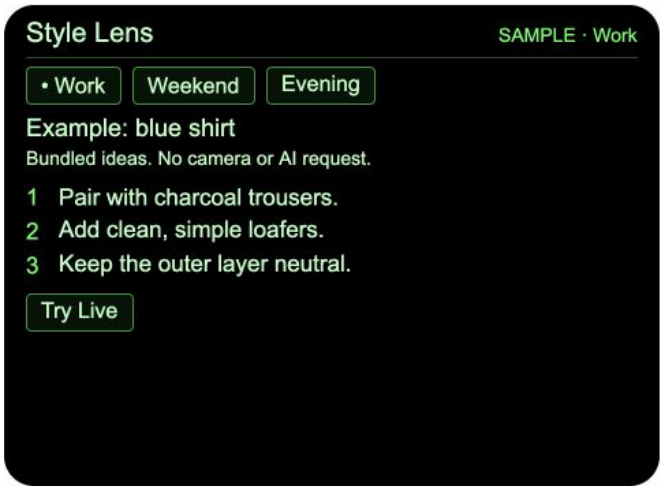


Try: choose coffee, move to Bloom, start the timer, pause and resume.

Watch the 30-second walkthrough

Style Lens

Occasion-based garment advice. Start with bundled Sample tips; no camera, server or API key.



Try: switch Work / Weekend / Evening, then Try Live and return to Sample.

Watch the 30-second walkthrough

Screenshots: actual patched Ink 0.19 browser diagnostics. Videos are edited screenshot walkthroughs. Neither is Studio or glasses footage.

> **Two repositories, two roles.** The Starter Kit contains editable AIUI apps. Your **Rokid AIUI Agent DevKit Skill** guides the coding assistant. Install the Skill once, then use it to modify either demo.

Before you begin

Bring a laptop, a coding tool with Agent Skills, **Node.js 22.20+**, **Python 3** and Git. Printed commands use a macOS/Linux shell; prepare an equivalent Windows environment before class. Studio use also requires access to AIUI Studio.

Open the Starter Kit

github.com/BreezeLife/rokid-aiui-starter-kit

Scan or click for source, multilingual READMEs and demo videos. Suggested hands-on loop: 5 min run, 15 min edit, 5 min check, 5 min show.

02 / INSTALL AND RUN

Get to a visible baseline

Use the repository root for terminal commands. Start with a no-key flow.

1. Get the project and install its tools

```
node --version
python3 --version
git clone https://github.com/BreezeLife/rokid-aiui-starter-kit.git
cd rokid-aiui-starter-kit
npm ci --ignore-scripts --no-audit --no-fund
```

2. Install and load your DevKit

```
npx skills add BreezeLife/rokid-aiui-agent-skill \
  --skill rokid-aiui-agent --agent codex --global
npx skills list --agent codex --global
```

Open this folder in your coding tool. The list should show **rokid-aiui-agent**. For Claude Code, replace `codex` with `claude-code` in both commands; for Cursor, use `cursor`.

> **In coding chat:** invoke `$rokid-aiui-agent` in Codex, or `/rokid-aiui-agent` in Claude Code. In Cursor, select the Skill from `/`. Ask the assistant to read the project instructions and report its loaded `SKILL.md` path.

3. Open the diagnostic browser walkthrough

```
npm run preview:diagnostic -- cook-along
npm run preview:diagnostic -- style-lens
python3 -m http.server 8768 --bind 127.0.0.1 \
  --directory output/previews
```

127.0.0.1:8768/cook-along-diagnostic.html
127.0.0.1:8768/style-lens-diagnostic.html

Keep the server terminal open; Ctrl+C stops it. The first generation needs internet to download the pinned runtime. After an edit, regenerate that demo and reload.

Using AIUI Studio?

Import only the folder containing `app.json`; open the Page in `_blank`.
Cook: `demos/cook-along/app` / Page `pages/index`
Style: `demos/style-lens/app` / Page `pages/index/index`

> **Preview status:** unmodified Ink 0.19 preview currently fails at startup. The diagnostic helper patches a separate local preview runtime. It supports layout/click practice; Studio and physical-glasses checks remain separate.

03 / MAKE IT YOURS

Change these three places first

Start with content, then adjust the interface. Keep each app self-contained.

Cook Along

Paths below are relative to `demos/cook-along/`.

1 / Content data

`app/lib/recipes.js`

Recipe names, short instructions and timer durations.

2 / UI copy

`app/pages/index.ink`

Headings, buttons and step layout.

3 / Agent prompt

`app/AGENTS.md`

Runtime Agent guidance and Page-opening behavior.

Style Lens

Paths below are relative to `demos/style-lens/`.

1 / Sample data

`app/data/samples.js`

Bundled tips. Keep the Sample label visible.

2 / UI copy

`app/pages/index/index.ink`

Occasion controls, headings and result layout.

3 / API config

`app/config.js`

Private endpoint + team token. Leave blank for Sample.

Copy this first exercise into your coding chat

```
$rokid-aiui-agent
Read this project's AGENTS.md and continuity files, then load the
installed Skill and report its path. In demos/cook-along/app, adapt
the coffee preset into a short tea-brewing guide. Keep the existing
recipe IDs, AIUI 0.19.0, English UI, step navigation and timer behavior.
Update the runtime guidance to match. Keep all app dependencies
inside its app directory. Run npm run verify; report changed files
and actual results. Keep untested Studio and glasses checks BLOCKED.
```

See your change: regenerate the chosen browser diagnostic, then reload it. In Studio, reimport your **edited local app directory**. For GitHub import, first push to your own repo/ref. Importing public `BreezeLife/main` again loads the original starter.

Your 30-second share-back: show the starting flow, one visible change, and the verification result. Each demo README contains three more copyable prompts. For Style Lens Live behavior, the model prompt is in `server/prompt.mjs`.

04 / VERIFY AND EXTEND

Check your change. Then go Live.

Sample mode is enough for the workshop. Live is an optional extension.

1. Verify from the Starter Kit root

```
npm run verify
npm run pack
```

verify runs tests, strict AIUI 0.19 checks and capability inventory. **pack** also requires Bash and writes archives to `output/packages/`. Local results: `output/verification/local-results.json`. Baseline: 38 passing tests; this does not certify Studio or glasses.

2. Optional: connect Style Lens to a vision provider

Keep Sample until your backend is ready. From the Starter Kit root, prepare the separate Node.js proxy:

```
cd demos/style-lens/server
cp -n .env.example .env
# Fill the private .env before starting (see below).
node --env-file=.env server.mjs
```

Set `VISION_API_KEY`, `VISION_MODEL`, `TEAM_TOKEN` and `TEAM_TOKEN_EXPIRES_AT`. The team token is 32-128 base64url characters; expiry must be within the next 24 hours. Generate fresh credentials near the session. Keep the provider key on the server.

Deploy behind reachable **HTTPS**; a laptop localhost URL is not a glasses endpoint. Follow the backend guide for token generation, health checks and Docker/Compose setup. Then edit these fields in your private `demos/style-lens/app/config.js`:

```
backendUrl: 'https://YOUR-HOST/api/style',
teamToken: 'YOUR_TEMPORARY_TEAM_TOKEN',
```

Verify from the root and reimport the edited app. Choose **Try Live, then Capture & send**, grant camera permission and confirm the system preview. This sends the photo to your configured provider and may incur charges. Keep the configured app private; **Use Sample** is the fallback.

If you get stuck

Skill missing: reload the tool; set `AIUI_SKILL_DIR` to the resolved Skill folder if needed.
Old UI still showing: regenerate the preview or check the imported local folder/repo/ref.
Live setup / 401: check the HTTPS endpoint, matching team token and fresh expiry.

> **Presenter preflight:** run both no-key flows on the teaching laptop before the session; keep the two 30-second videos available. Studio, glasses, real camera/provider and Docker execution remain unverified in this starter. Use Sample unless you have tested your own Live setup.